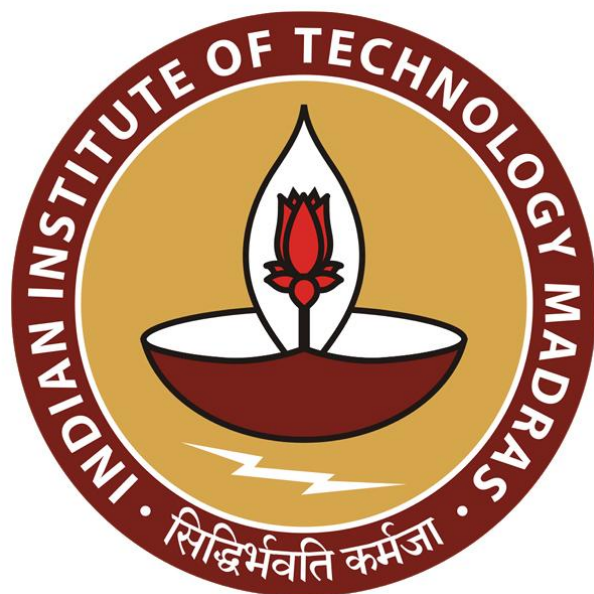
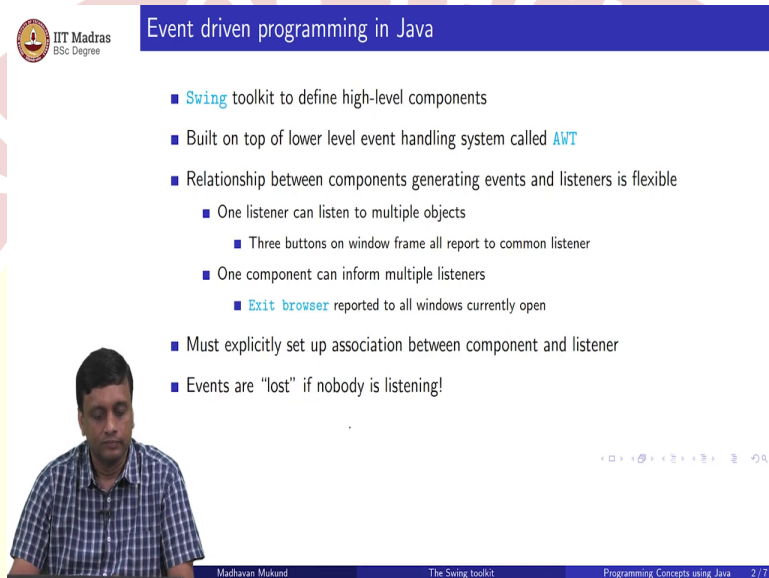




IIT Madras
ONLINE DEGREE

Programming Concept using JAVA
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
The Swing Toolkit

(Refer Slide Time: 00:15)



The screenshot shows a presentation slide with a blue header titled "Event driven programming in Java". The slide contains a list of bullet points explaining the Swing toolkit and event-driven programming. In the bottom left corner, there is a small video inset showing Professor Madhavan Mukund. The bottom of the slide features a navigation bar with the text "Madhavan Mukund", "The Swing toolkit", "Programming Concepts using Java", and "2 / 7".

- **Swing** toolkit to define high-level components
- Built on top of lower level event handling system called **AWT**
- Relationship between components generating events and listeners is flexible
 - One listener can listen to multiple objects
 - Three buttons on window frame all report to common listener
 - One component can inform multiple listeners
 - **Exit browser** reported to all windows currently open
- Must explicitly set up association between component and listener
- Events are "lost" if nobody is listening!

So, we saw that event driven programming is the way to go if we want to design graphical user interfaces. So, let us look at event driven programming in Java. So, event driven programming in Java is implemented using something called Swing. So, this is a toolkit, as it is called it. So, you can think of it just as a library or a package.

So, it is a package of objects and various other functionalities, which allow us to do this kind of graphical user interface programming using events and listeners. So, swing is actually built on top of a more abstract event handling framework called AWT, which stands for Abstract Window in Toolkit. So, AWT is a low-level thing which manages events and interfaces. And you can also use it to design customized events, which we will not discuss in this part of the course.

But Swing is built on top of that, and swing has specific types of events, which are associated with specific listeners, as we will see. So, the most important feature about this in Java is that this relationship between components which generate events and listeners is very flexible. We

talked about the timer and said that in our example that we did callbacks, we said that the object that created the timer, in our example was the one that was notified when the timer elapsed.

But we could have a more general thing which says that the object that creates a timer passes a reference to another object to be notified when the timer elapses. And similar things happen in graphical user interfaces. So, we could have a common listener listening to multiple objects. So, for instance, if we have three different windows, buttons on a window, we might want to do something to the underlying window depending on the button.

So, the same listener must listen to all three buttons so that there is no confusion about what should be done when different buttons are pressed. On the other hand, we could have one button talking to multiple listeners. So, for instance if we have some kind of global quit, or logout or a restart, so this must close every window which is available on the system. Or if we just want to quit an application which has opened multiple windows, we want to close all the windows, so we do not want to go around individually clicking them.

So, we click this one button, then all of those windows need to be notified. So, in the first example, we had one window listening to multiple buttons, here you have multiple windows listening to one button. So, we can have many event generators talking to one listener, and you can have one event generator talking to multiple listeners.

So, this means that we need to in Java explicitly set up this connection, it is not that every time we create an object, which has some capability of generating events like a window or a button, automatically listener is created, we have to separately create the appropriate listener for our requirement and set up this correlation as we said that add to a given object which generates events, we have to tell it, which listener is the one that will be notified. So, this manual thing is required.

And therefore, a side effect of that in Java is that if you do not do that, the object that is generating events will just keep on generating them and nobody is listening. So, sometimes we have I mean, when our application is not working properly, we click on something and nothing happens, we want to close the application the application is unresponsive. So, one of the reasons

an application could be unresponsive is that the thread which is supposed to be listening to that event has unexpectedly quit. So, there is nobody listening. So, all your clicks have no effect because there is nobody on the other side receiving those signals.

(Refer Slide Time: 03:53)

A button that paints its background red

- `JButton` is Swing class for buttons
- Corresponding listener class is `ActionListener`
- Only one type of event, button push
 - Invokes `actionPerformed(...)` in listener

Diagram: A handwritten note shows 'Button' and 'Press' (circled in red) with arrows pointing to 'ActionListener' and 'actionPerformed' respectively.

Madhavan Mukund | The Swing toolkit | Programming Concepts using Java | 3/7

So, let us start looking concretely at some examples of how Swing does this. So, buttons as we said are simple object which you can press. So, in Swing, there is a built-in class called `JButton`. So, in general, these Swing objects have this capital J at the beginning. And without the J these correspond to things which are defined in the underlying AWT class, which we will not normally deal with.

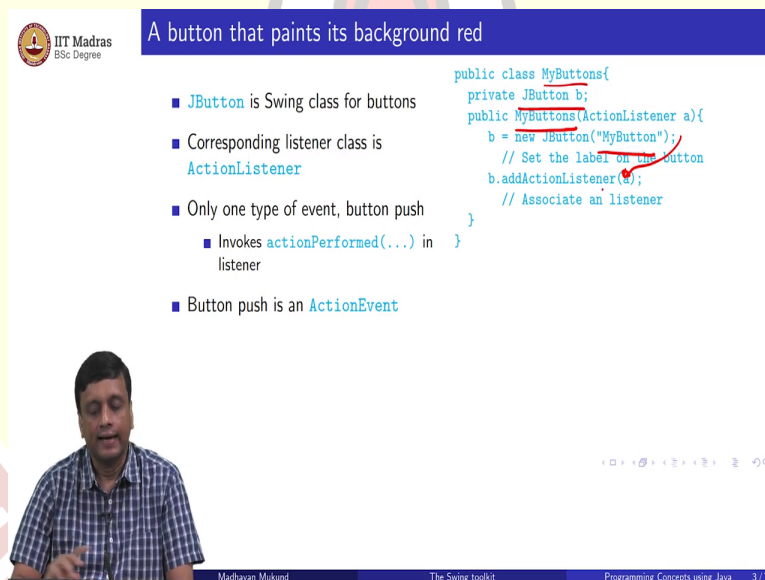
So, whenever you see a Swing component, you expect to see a capital J in front of it. So, `JButton` is a built-in class. It is a button. So, we do not have to define anything more about it because it is given to us, it is a predefined class. And a `JButton` as defined talks to something called an `ActionListener`. So, remember that we said that a timer talks to something called a timer owner, this is a predefined connection saying which interface is the one that is notified when a timer elapses.

So, when a button is pressed, the kind of interface that I need the kind of implementation that I need to talk to is something that looks like an `ActionListener`. Now in a button, there is not much

you can do, as we discussed, you can just press it. So, therefore the ActionListener essentially has a unique function, that is called when the action happens. And that is called actionPerformed. So, in other words, when you press a button, so you get a button and you press it. And what happens is that there is a corresponding ActionListener.

And in that ActionListener, you perform the implementation of actionPerformed. So, this whole thing happens automatically. So, the runtime system, make sure that when this happens, this happens. So, you do not have to make this connection after you set it up initially. Initially, you say, this button talks to this ActionListener, that is all you have to say. And you of course, have to provide the code on both sides. You have to say what the button looks like, and what the ActionListener is going to do when it gets a signal.

(Refer Slide Time: 05:57)



A button that paints its background red

- JButton is Swing class for buttons
- Corresponding listener class is ActionListener
- Only one type of event, button push
 - Invokes actionPerformed(...) in listener
- Button push is an ActionEvent

```
public class MyButtons{
    private JButton b;
    public MyButtons(ActionListener a){
        b = new JButton("MyButton");
        // Set the label on the button
        b.addActionListener(a);
        // Associate an listener
    }
}
```

Madhavan Mukund The Swing toolkit Programming Concepts using Java 3/7

So, what the button push generates is an object of type ActionEvent. So, the events themselves, like everything else in Java, have structured because we might want to know, as we will see later, something about the event. So, this generates an object called an ActionEvent, which is then analyzed by this actionPerformed. So, here is a kind of simple way to do this. So, you first set up a button. So, you say that, I have a JButton, which I will call b.

And I will now create ActionListener, sorry, this is a constructor, which says that, when I create a JButton in this class, I will pass to it an ActionListener. And then what this will do is it will set up the new button, and it will add the ActionListener that is passed to that button saying this button is now connected to these particular actions.

So, when I create a button, this is what I need to do, I need to set up something. So, here, for instance, this is a property of the button, what will the button display. So, it displays the word MyButton. And what will the button talk to when it is pressed, that is given by this addActionListener.

(Refer Slide Time: 07:13)

A button that paints its background red

- JButton is Swing class for buttons
- Corresponding listener class is ActionListener
- Only one type of event, button push
 - Invokes actionPerformed(...) in listener
- Button push is an ActionEvent

```
public class MyButtons{
    private JButton b;
    public MyButtons(ActionListener a){
        b = new JButton("MyButton");
        // Set the label on the button
        b.addActionListener(a);
        // Associate an listener
    }
}

public class MyListener implements ActionListener
public void actionPerformed(ActionEvent e){...
    // What to do when a button is pressed
}

public class XYZ{
    MyListener l = new MyListener();
    // ActionListener l
    MyButton m = new MyButtons(l);
    // Button m, reports to l
}
```

Madhavan Mukund The Swing toolkit Programming Concepts using Java 3/7

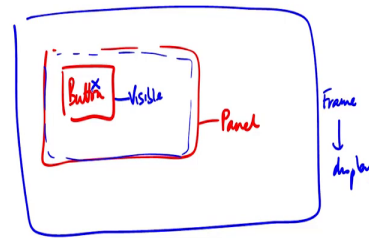
Now, on the other side, I must create this ActionListener. So, I will create a class, which has an implementation of this function actionPerformed, because that is what the ActionListener interface asks for. So, I will create a separate class with justice. So, now in practice, what I will do is I will first create a new object which can listen, I will then create a new button and to this button, I will pass the object that can listen so that that button b, or sorry, this button m is now connected to this listener l. So, this is an abstract example, we will look at a little more concrete example in a minute.

(Refer Slide Time: 07:50)



Embedding the button in a panel

- To actually display the button, we have to do more



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java 4/7

So, this just sets up a button. Now unfortunately, we just set up a button. It is not displayed. So, displaying a button is a different thing altogether. So, Java has a notion called a frame. And a frame is what can be displayed. So, we ultimately have to create a frame and make it visible. Otherwise, these things are just happening in the background. So, although there is a button that the user can press, the user cannot see that button, and therefore cannot interact with it.

So, what we have to do is actually a two-step process. So, we have to take the button, and we have to put it somewhere. So, there will be a button. So, it will set somewhere. And that somewhere, is what Java calls a panel. And then this panel, we will have to go into something bigger, with Java called the frame. And this can be displayed.

So, when I display the frame, it will display this panel, this panel boundary is typically an abstract boundary, I mean, it need not have a boundary that is visible. But in that there will be a visible button, which the user can interact with. So, now if I come to this button and I press on it, then automatically the appropriate listener will be invoked. But if I cannot see this button, I cannot press it. So, that is why we have to go through this extra level of setting up this display.

(Refer Slide Time: 09:06)



Embedding the button in a panel

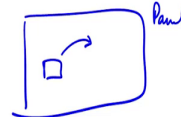
- To actually display the button, we have to do more

- Embed the button in a panel — `JPanel`

- First import required Java packages
- The panel will also serve as the event listener

```
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;
```

```
public class ButtonPanel extends JPanel {  
    implements ActionListener {  
        ...  
    }  
}
```



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java 4/7

So, we have to put this in a panel. So, not surprisingly, given what we talked about with button, a panel in Swing is called a JPanel because everything in Swing normally starts with a capital J. So, we first import some useful packages that we need for Swing. So, remember, it has this underlying AWT, and this is the Swing thing. So, what we need to do is create this ButtonPanel. So, this ButtonPanel will be a subclass of JPanel.

And this panel will So, remember where the panel is. So, we have this panel. And we have a button b inside. So, this button has to talk to somebody. So, what we are going to do is make a talk to the enclosing panel. So, the panel will not only contain the button, but the panel will also implement the function that responds to the button.

(Refer Slide Time: 10:04)



Embedding the button in a panel

- To actually display the button, we have to do more
- Embed the button in a panel — JPanel
 - First import required Java packages
 - The panel will also serve as the event listener
 - Create the button, make the panel a listener and add the button to the panel

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ButtonPanel extends JPanel
    implements ActionListener{

    private JButton redButton;

    public ButtonPanel(){
        redButton = new JButton("Red");
        redButton.addActionListener(this);
        add(redButton);
    }
}
```

Implementation of ActionListener



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java 4/7

So, the first thing is to create a button. So, here I declare a variable of type JButton. And now I create in this ButtonPanel, when I create a panel, I create a new button. In this case, I am going to give it a label Red, then I am going to add this current panel as an ActionListener, I have not yet told you how this ActionListener is going to respond, that is going to be here. So, this is going to be the implementation of this ActionListener part.

So, that is yet to come. But this class already knows how to respond to ActionListener events. So, we say that I have a red button, which has a label Red, it is going to talk to the enclosing panel that is this object. And then having set up this button, I add that button to this panel. So, this just creates a button and puts it physically in this panel. So, I now embedded this button in this panel. So, I still have to talk about what this ActionListener does.

(Refer Slide Time: 11:06)



Embedding the button in a panel

- To actually display the button, we have to do more
- Embed the button in a **panel** — `JPanel`
 - First import required Java packages
 - The panel will also serve as the event listener
 - Create the button, make the panel a listener and add the button to the panel
- Listener sets the panel background to red when the button is clicked

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ButtonPanel extends JPanel
    implements ActionListener{
    private JButton redButton;

    public ButtonPanel(){
        redButton = new JButton("Red");
        redButton.addActionListener(this);
        add(redButton);
    }

    public void actionPerformed(ActionEvent evt){
        Color color = Color.red;
        setBackground(color);
        repaint();
    }
}
```



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java

4/7

So, the listener, in this case, is going to color the panel according to the color which I put on the label Red. So, what the `actionPerformed` in this case is it sets the background colour to a built-in color, which Swing has called Red. And then it repaints the panel, so it makes the panel background red. So, the effect of clicking this button is to take the panel that is there and make it red. Earlier, it was probably not red. And now it is red so that is the effect that you are going to see.

(Refer Slide Time: 11:38)

Embedding the panel in a frame

- Embed the panel in a frame —
JFrame

```
public class ButtonFrame extends JFrame
    implements WindowListener {

    public ButtonFrame() { ... }

    // Implement WindowListener

}
```



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java

5/7

Now, remember that the panel itself needs to be sitting in a frame because the frame is what Java can display. So, we need to create something, which is of type JFrame. So, remember JButton, JPanel now JFrame. So, the button is embedded in the panel, and the panel is embedded in the frame, and the frame is what Java can actually display. So, we will have now a constructor.

But now it turns out that the frame itself what I display, generate its own events, why? Because this is what we see when we open a window. That is the one which has the terminate full screen, and minimize and so on. So, every frame comes with its own events, independent of the events that are generated by the objects that are independent. So, in order to implement a button frame, I need to connect it, I mean, to implement a JFrame, I need to connect it to a listener. So, like in the panel, we are going to make this frame talk to itself.

(Refer Slide Time: 12:42)

सिद्धिर्भवति कर्मजा

Embedding the panel in a frame

- Embed the panel in a **frame** — **JFrame**
- Corresponding listener class is **WindowListener**
- **JFrame** generates seven different types of events
 - Each of the seven events automatically calls a different function in **WindowListener**
- Need to implement **windowClosing** event to terminate the window
- Other six types of events can be ignored

```
public class ButtonFrame extends JFrame
    implements WindowListener {

    public ButtonFrame(){ ... }
    ...
}

// Six of seven methods required for
// implementing WindowListener are stubs
public void windowClosing(WindowEvent e) {
    System.exit(0);
}
public void windowActivated(WindowEvent e){}
public void windowClosed(WindowEvent e){}
public void windowDeactivated(WindowEvent e){}
public void windowDeiconified(WindowEvent e){}
public void windowIconified(WindowEvent e){}
public void windowOpened(WindowEvent e){}
```



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java 5/7

So, the corresponding class for a frame is a **WindowListener** as opposed to the **ActionListener** that we had for a button. So, it turns out that frames can generate seven different kinds of methods, I mean seven different kinds of events, but fortunately, we are not obliged to actually implement all of them except the one which corresponds to closing.

So, the only thing that we must have an implementation for is the function that is to be called when you press the button which asks you to quit the application. So, this is the function that we end up calling when you quit the application the one you have when you press X is called **windowClosing**. So, we have to provide an implementation of **windowClosing**.

And so here is a suitable thing which just says exit this program. Now, there are all these other things like activating, closing, deactivating, deiconified, iconified and all that. So, you can provide suitable implementations as to what to do with that visible frame and all of these things are invoked, but it is allowed to just make them do nothing. So, this is like having an unresponsive button, you press it, nothing happens, you want to make this thing full screen, nothing happens, you want to iconified, nothing happens, you press X, it dies, that is the only thing that we are obliged to.

So, the other 6 types of events can be ignored. So, what we have done is we have created a frame in which we are going to put the panel we have not yet done that. But first thing we have to do is

that frame itself has to have a corresponding listener. So, we are making this frame talk to itself, the frame implements its own listener, just like the panel implements the listener for the button that it encloses.

(Refer Slide Time: 14:21)

Embedding the panel in a frame

- One more complication
- JFrame is "complex", many layers
- Items to be displayed have to be added to `ContentPane`

```
public class ButtonFrame extends JFrame
    implements WindowListener {

    Private Container contentPane;

    public ButtonFrame(){
        setTitle("ButtonTest");
        setSize(300, 200);

        // ButtonFrame listens to itself
        addWindowListener(this);

        // ButtonPanel is added to the contentPane
        contentPane = this.getContentPane();
        contentPane.add(new ButtonPanel());
    }

    // Six of seven methods required for
    // implementing WindowListener are stubs
}
```

Madhavan Mukund The Swing toolkit Programming Concepts using Java 6 / 7

Now, there is one more minor complication. So, a frame is not just a kind of single level object, it is a multi layered object. So, it has, so a frame abstractly has multiple layers associated so that you can separate out different parts of it. So, what we are interested in is the layer on which we put the things that are visible, and that is called the `ContentPane`. So, complex `JFrame` has one part called the `ContentPane`, which is where the visible objects that we add to the frame are to be defined.

So, at the bottom here, we have suppressed now the code for those 7 listener interface functions. And instead, we are focusing on now what the frame contains. So, first of all, we have to extract its `ContentPane`. And now we say that when we create a frame, we set its title and its size. So, it is going to have a thing saying button test. And it is going to have a 300 by 200 size in terms of the geometry of the window, 300 pixels by 200 pixels.

We are going to say that this button frame talks to itself. It has its own implementation for all the events that it generates. And now finally, I add that panel which I created. So, I get this

ContentPane, and I add a new ButtonPanel. So, ButtonPanel, remember we just defined is the one that then in turn creates every button called Red, and it implements a function, when Red is pressed, turn the panel background to red. So, that is it.

(Refer Slide Time: 15:52)



Finally, a main function

- Create a `JFrame` and make it visible

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ButtonTest{
    public static void main(String[] args) {
        EventQueue.invokeLater(
            () -> {
                JFrame frame = new ButtonFrame();
                frame.setVisible(true);
            }
        );
    }
}
```



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java

7/7

So, this is finally we have got this. Then, of course, we have to write a main program, which creates the frame and makes it visible. So, what we have done is we have set up this frame, which contains a panel which contains a button, now we have to create a frame and make it visible. So, essentially we have to do this, we have to create a new button frame and set visible to true. That is the way that we make it visible on the screen. Otherwise, by default, when you create a frame, it is not displayed.

(Refer Slide Time: 16:18)



Finally, a main function

- Create a `JFrame` and make it visible
- `EventQueue.invokeLater()` puts the Swing object in a separate event dispatch thread

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ButtonTest{
    public static void main(String[] args) {
        EventQueue.invokeLater(
            () -> {
                JFrame frame = new ButtonFrame();
                frame.setVisible(true);
            }
        );
    }
}
```



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java 7/7

Now, one small thing that happens here is that we have to hide this whole code inside this function called `invokeLater`. And this is a technicality, which is basically to do with the fact that everything in this is running in a threaded mode. Remember that we said that the graphical user interfaces is kind of interacting with the user separately from what is happening in the rest of the thing. So, you have to be responsive to clicks and all that while you are doing whatever you are supposed to be doing.

So, in order to make these things work properly, we have to put all the Swing components which interact with the user into a separate thread, and that is what this, so you just have to remember to do this, you do not have to change it ever, wherever you want to create some frame, you have to put it into this `EventQueue.invokeLater()`. And this is a format which uses the lambda expression, there is also an explicit way of doing it, you can look it up. So, what this does is it puts this frame into something called an event dispatch thread.

(Refer Slide Time: 17:13)



Finally, a main function

- Create a `JFrame` and make it visible
- `EventQueue.invokeLater()` puts the Swing object in a separate **event despatch thread**
- Ensures that GUI processing does not interfere with other computation
- GUI does not get blocked, avoid subtle synchronization bugs

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ButtonTest{
    public static void main(String[] args) {
        EventQueue.invokeLater(
            () -> {
                JFrame frame = new ButtonFrame();
                frame.setVisible(true);
            }
        );
    }
}
```



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java 7/7

And this ensures that the processing of the interactions, the button presses and all these interactions that the interface generates do not contradict what is happening, so you do not get blocked. So, the computation does not get sort of held up because, or the interface does not get held up because the computation is doing something heavy. If they were running in the same thread, then you have a problem that you have to manually manage the listening to. So, for example, you are downloading a large file in a browser and there is a stop button.

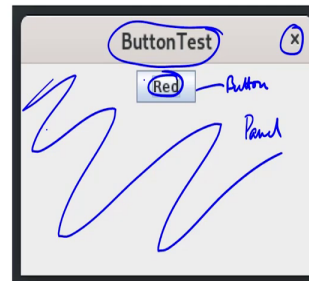
So, if you are going to interleave how much you download before you check for the stop button then you are controlling the rate at which the stop is responded to. Whereas if the stop is parallel to the download, which is what is going to happen here because the stop will be part of the GUI and the download will be in a different thread, then the stop will be more responsive because it is not really directly connected to how long or how heavy the download process is.

(Refer Slide Time: 18:09)



Finally, a main function

- Create a `JFrame` and make it visible
- `EventQueue.invokeLater()` puts the Swing object in a separate *event dispatch thread*
- Ensures that GUI processing does not interfere with other computation
- GUI does not get blocked, avoid subtle synchronization bugs
- Output — before the button is clicked



Madhavan Mukund

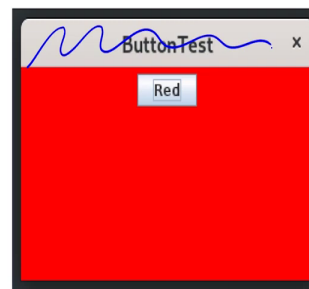
The Swing toolkit

Programming Concepts using Java 7/7



Finally, a main function

- Create a `JFrame` and make it visible
- `EventQueue.invokeLater()` puts the Swing object in a separate *event dispatch thread*
- Ensures that GUI processing does not interfere with other computation
- GUI does not get blocked, avoid subtle synchronization bugs
- Output — before the button is clicked
- ... and after



Madhavan Mukund

The Swing toolkit

Programming Concepts using Java 7/7

So, anyway, you set this all up and this is what you get. So, when you first run this code, so you have this, this main function, which calls the `JFrame`, which calls the `JPanel`, which includes a `JButton` in it, you see this kind of a window. So, remember that we created the frame with this title button test. It has an X built into it because it always has an X. And this is the panel. So, this part is the panel, implicitly.

So, the panel is embedded in the frame, and inside the panel, this is our button. And when we created the button, we put this label Red on it. So, that is all we see. Now, we see a button with

the label Red because we created a button, the panel created a button inside it with the label Red and the frame, embedded this panel and put a frame title called ButtonTest. So, now if I click on this button, what happens is that the panel turns red. So, the surrounding area outside the button, which is the background of the panel, turns red. And notice that this header part of the frame is not part of the panel. So, this title window is separate. So, only the bottom parts turns.

(Refer Slide Time: 19:20)

Summary

- The Swing toolkit has different types of objects
- Each object generates its own type of event
- Create an appropriate event handler and link it to the object
- The unit that Swing displays is a frame
- Individual objects have to be embedded in panels which are then added to a frame

Madhavan Mukund The Swing toolkit Programming Concepts using Java 8 / 7

So, what we have seen is a very simple example of how we program a graphical object in Swing. So, we will see that Swing has many different types of objects. So, each object will generate its own type of event. And we have to now create the corresponding handler object, the event handler, which will listen to these events and respond to them.

And what we finally have to do is put everything in a frame because swing will only display frames. So, we have to put individual objects into panels. And then we have to put these panels into frames. So, we have to do a little bit of work to get all these listeners moving and all these things embedded in each other. And finally, we can create that even more complicated in that thread section, we have to create a new frame and then make it visible.